



A Comparative Analysis of Traditional Models and Deep Learning for Value at Risk Estimation

Final Capstone Project Report

Group 5

MSDSM01

Submitted by:

Georgy S Pulayath - MSDSM/01/012

Joshi Bhavin Rasikbhai - MSDSM/01/016

Prabhujeet Sahu - MSDSM/01/023

Shrenik Nandre - MSDSM/01/032

Under the guidance of

Dr. Puneet Pasricha

(IIT Ropar)

1. Introduction

1.1 Problem Statement

With increasing globalization and rapidly changing world influencing the market volatility, financial institutions require reliable risk estimation techniques. The primary goal of Value at Risk (VaR) estimation is to quantify the potential loss in a financial portfolio over a given time period with a specified probability. While traditional models such as Historical Simulation, Monte Carlo and GARCH have been widely used, they fail to fully capture complex, non-linear dependencies in financial data.

In this project, we are trying optimize model dynamics for VaR forecasting using deep learning methods, particularly utilizing Encoded VaR by leveraging Variational Autoencoders (VAEs). This study seeks to compare the performance of traditional methods and ML-based VaR estimation methods, identifying their strengths and weaknesses while exploring hybrid approaches that combine both paradigms.

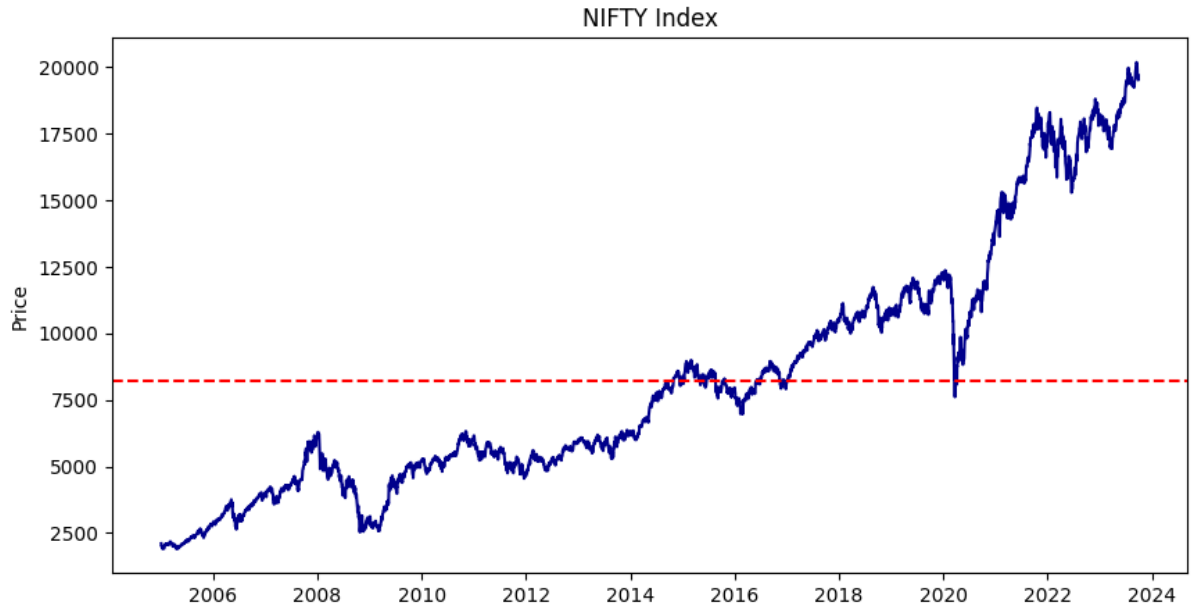
1.2 Significance

Since the traditional risk estimation models assume static market conditions and normal distributions, these traditional models are less effective in dynamic financial environments. Encoded VaR, on the other hand, can adaptively learn from historical data, capturing non-linear market behaviours. This research contributes to financial risk management by:

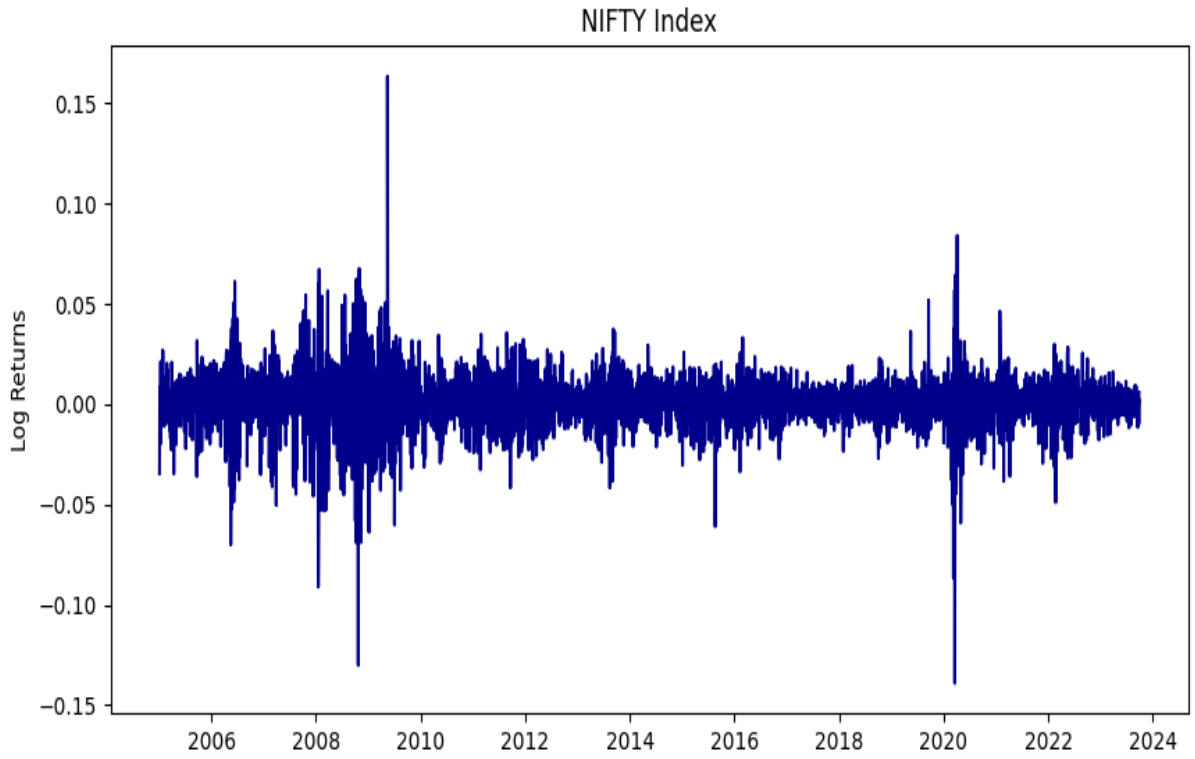
- Demonstrating where machine learning outperforms traditional methods in VaR estimation.
- Evaluating the real-world applicability of Encoded VaR through empirical testing.
- Exploring hybrid methodologies that enhance risk estimation accuracy.

2. Dataset Description

For the demonstration purpose considering the computing resources constraint, we have used Nifty50 index data for the modelling purposes. The index values are from date January 4, 2005 to September 25, 2023.



Following graph show log returns distribution for Nifty50 index over the given time period.



As we can clearly see from the graph there is evidence of volatility clustering in several smaller time periods, the ARMA/ARIMA may not give true model for the volatility estimation and hence these models are not meaningful to estimate Value at Risk (VaR) and back test.

3. Historical Simulation Approach

3.1 Introduction

The Historical Simulation Method is a widely used non-parametric approach for estimating Value at Risk (VaR). In this method, we estimate VaR using past observed returns to estimate potential future losses without assuming a specific distribution. This method is particularly useful in financial risk management as it provides an empirical distribution of losses based on historical data.

Unlike parametric models, such as the variance-covariance approach, the historical simulation method does not require assumptions about return distributions. Instead, it directly utilizes past returns to construct a loss distribution. This method is computationally straightforward and adapts well to different market conditions, making it a fundamental choice for many financial institutions to set benchmark for other estimation and forecasting models.

3.2 Methodology

For the purpose of illustration, we have considered the portfolio worth 1 crore INR in Nifty Index starting from Day 1. The historical simulation method follows a structured process to estimate VaR:

- Collect Historical Returns: A dataset of past 501 days of financial returns is gathered. This provides 500 alternative scenarios for predicting future losses.
- Generate Scenarios: In this method, we generate 500 scenarios using the 501 days of financial. The first scenario is the returns generated in our portfolio by changes in Nifty

Index. The second scenario is daily returns between day 3 and day 2 in our portfolio and continuing this way 500th scenario is the returns generated through our portfolio with respect to change in index value.

- Sort Returns in Ascending Order: Arrange the returns obtained by generating scenarios from the most negative to the most positive for the given 500 scenarios. This ranking helps identify extreme loss scenarios.
- Determine the Percentile Cutoff: Define the confidence level (e.g., 95% or 99%) and select the corresponding quantile from the sorted returns. For example, in a dataset of 500 generated scenarios, the 5th percentile (for 95% confidence) is at the 25th worst return and one percentile is 5th worst return. Here, we have considered 1% confidence interval.
- Estimate VaR: The chosen quantile value represents the potential loss threshold for the given confidence level.

3.3 Advantages and Limitations

Advantages:

- ✓ No Distributional Assumptions: The method is model-free, avoiding issues with normality assumptions in parametric models.
- ✓ Simple Implementation: It only requires historical data and basic statistical operations.
- ✓ Captures Market Realities: Since it directly uses past returns, the method inherently accounts for historical volatility and extreme market movements.

Limitations:

- ☒ Limited to Past Data: The method assumes that future market conditions will resemble historical trends, which may not always hold.

- ☒ Data Dependency: A small dataset may lead to unreliable estimates, while a large dataset may include outdated market conditions.
- ☒ Inability to Capture New Risks: This method does not consider emerging risks that have not occurred in the past data window.

3.4 Empirical Application

In this study, the Historical Simulation Method is applied to Nifty 50 index data to estimate VaR. The methodology follows these steps:

- Data Collection: Historical price data of Nifty50 is used to compute daily returns.
- Sorting and Quantile Selection: Returns are sorted in ascending order, and the appropriate percentile is chosen for the VaR estimation.
- VaR Calculation: The identified return at the given confidence level represents the estimated VaR.

3.4 Results:

The AE ratio has the mathematical formulation as:

$$AE\ Ratio = \frac{Number\ of\ Actual\ Violation}{Number\ of\ Expected\ Violations}$$

AE ratio gives a very practical understanding of model robustness with AE ratio close to 1 indicating model is good and AE ratio far away from 1 indicates that model is not able to capture underlying pattern and trend of financial data. Here when we say model, it is true for every VaR forecasting and backtesting model, not limited to Historical Simulation. The value of AE ratio for the Historical method approximately 3.4.

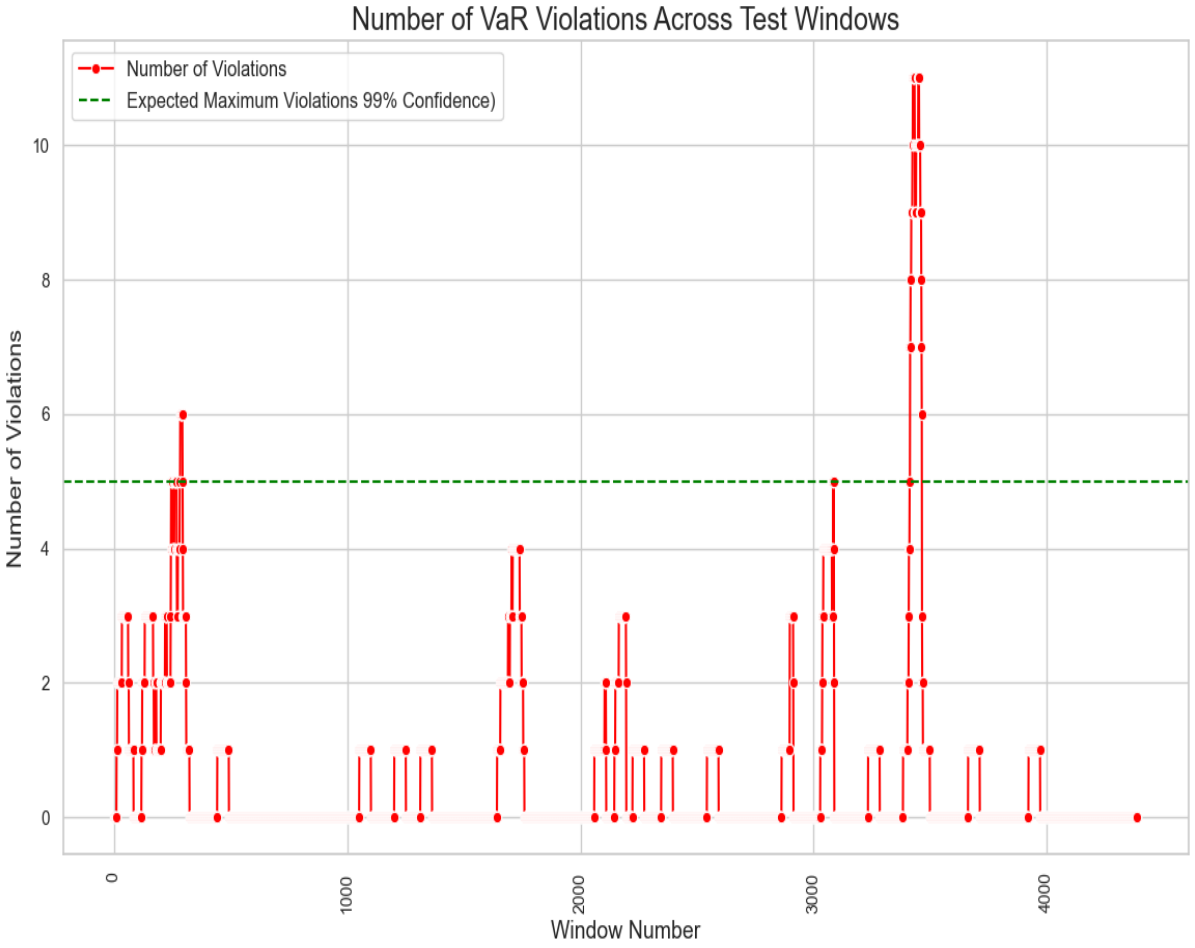
```
# Compute the AE Ratio
AE_ratio = total_violations / expected_violations if expected_violations > 0 else None


print(f"Total Expected Violations: {expected_violations:.2f}")
print(f"Actual Exceedance (AE) Ratio for Historical Model: {AE_ratio:.4f}" if AE_ratio is not None else "AE Ratio cannot be calculated.")
```

```
[82] ✓ 0.0s

Total Expected Violations: 750.00
Actual Exceedance (AE) Ratio for Historical Model: 3.4307
```

The following graph shows the distribution of violation over the different testing windows.



The results from Unconditional coverage test, independence test and conditional coverage test have p-value approximately 0.998 indicating model is correctly calibrated and violations occur independently over time. Hence from these tests it is evidence of misspecification of models

and model reliable is for risk estimation, though AE ratio is far from 1. But it can give satisfactory results. Overall, The Historical Simulation Method provides an intuitive and practical approach for VaR estimation. Its reliance on actual market data makes it robust, but its effectiveness depends on the availability of a comprehensive dataset. While it is widely used in financial risk management, supplementing it with stress testing and scenario analysis can enhance its reliability.

```
# Perform Unconditional Coverage Test (Kupiec's POF Test)
print("\nPerforming Coverage Tests for Historical model:")
pof_lrt, pof_pval = kupiec_pof_test(total_violations, total_days, sig)
print(f"Kupiec's POF Test: LR Statistic = {pof_lrt:.4f}, p-value = {pof_pval:.4f}")

# Perform Independence Test (Christoffersen's Test)
ind_lrt, ind_pval = christoffersen_independence_test(all_violations)
print(f"Christoffersen's Independence Test: LR Statistic = {ind_lrt:.4f}, p-value = {ind_pval:.4f}")

# Perform Conditional Coverage Test
cc_lrt, cc_pval = christoffersen_conditional_coverage_test(pof_lrt, ind_lrt)
print(f"Christoffersen's Conditional Coverage Test: LR Statistic = {cc_lrt:.4f}, p-value = {cc_pval:.4f}")
```

✓ 0.0s

```
Performing Coverage Tests for Historical model:
Kupiec's POF Test: LR Statistic = -0.0000, p-value = 1.0000
Christoffersen's Independence Test: LR Statistic = -0.0000, p-value = 1.0000
Christoffersen's Conditional Coverage Test: LR Statistic = -0.0000, p-value = 1.0000
```

4. Single Asset Case Approach

4.1 Introduction

Single Asset case is a simple mathematical approach to estimate and backtest Value at Risk (VaR), where the portfolio consists of 1000 positions that is a portfolio worth ₹2103750.0 on January 4th, 2005 in the Nifty50 index. The objective is to determine the 1-day VaR, and check the robustness of model with parameters like AE ratio and coverage tests.

4.2 Assumptions and Parameters

- **Portfolio Position:** ₹2103750.0 invested in the Nifty 50 index with 1000 positions.

- **Time Horizon:** 2005 to 2023.
- **Confidence Level:** 95%.
- **Volatility of Nifty 50:** 1% per day (approximately 15.87% per year). (in initial years)
- **Distribution Assumption:** The daily returns of the Nifty 50 index follow a normal distribution with an expected daily return of zero.
- **Z-score for 95% confidence level:** 1.645 (from standard normal distribution tables).

4.3 1-Day VaR Calculation

The daily standard deviation of the portfolio value is calculated as:

Applying the VaR formula:

Thus, the **1-day 95% VaR** for the portfolio is **₹50474**, meaning that under normal market conditions, there is a 95% probability that losses will not exceed ₹50474 in a single trading day. The mathematical formulation is:

$$daily\ volatility = 1.09\%$$

The portfolio value on the day before forecasting is ₹2032200.0. Hence,

$$standard\ deviation\ of\ daily\ position = 0.0109 * 2032200.0 = 22150.98$$

Hence 1-day VaR will be

$$1 - day\ VaR = 22150.98 * 1.645 = 36447.40$$

4.4 Backtesting and Results

For the backtesting we have used AE ratio which on average have the value of 2.33.

```
mean_ae_ratio = np.mean([x for x in ae_ratio_list if x != 0])
print(f"Average AE ratio over all testing windows is:{mean_ae_ratio}")
```

✓ 0.0s

```
Average AE ratio over all testing windows is:2.3298597194388777
```

Other than that, following tests have been used:

1. **Historical Testing:** Checking if the actual losses exceed the VaR threshold in less than 5% of cases.
2. **Independence Test:** Ensuring violations are independent rather than clustered.
3. **Conditional Coverage Test:** Using statistical tests to confirm that observed violations occur independently without overtime certainly below given threshold.

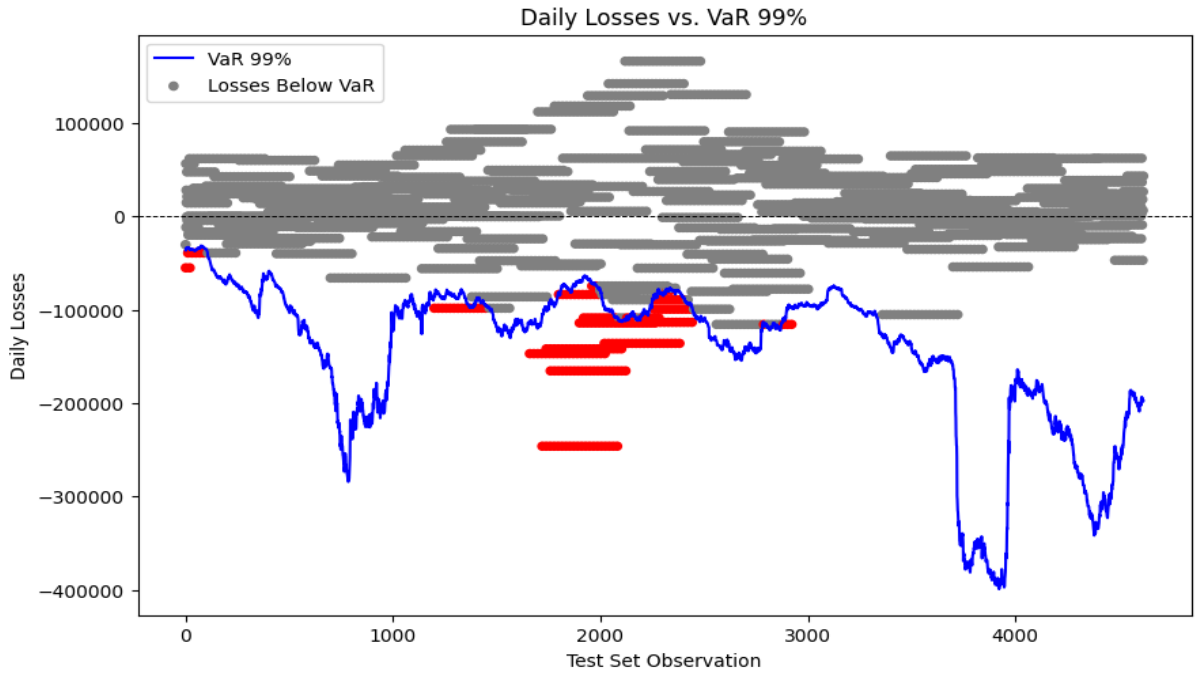
```
# Perform tests
total_observations = len(violation_sequence)
total_violations = sum(violation_sequence)

kupiec_stat, kupiec_p = kupiec_test(total_violations, total_observations)
ind_stat, ind_p = christoffersen_test(violation_sequence)
joint_stat, joint_p = conditional_coverage_test(total_violations, total_observations, violation_sequence)
print("Following are the results of the coverage tests for Single Asset Case model:")
print(f"Kupiec Unconditional Coverage Test: Chi-Square Statistic = {kupiec_stat:.4f}, p-value = {kupiec_p:.4f}")
print(f"Christoffersen Independence Test: Chi-Square Statistic = {ind_stat:.4f}, p-value = {ind_p:.4f}")
print(f"Conditional Coverage Test: Chi-Square Statistic = {joint_stat:.4f}, p-value = {joint_p:.4f}")
```

✓ 0.0s

```
Following are the results of the coverage tests for Single Asset Case model
Kupiec Unconditional Coverage Test: Chi-Square Statistic = 0.0000, p-value = 1.0000
Christoffersen Independence Test: Chi-Square Statistic = 0.0000, p-value = 1.0000
Conditional Coverage Test: Chi-Square Statistic = 0.0000, p-value = 1.0000
```

p-values are near to 1.0 which indicates that the violations occur independently over time with majorly being under the threshold level. Following graph supports the given argument.

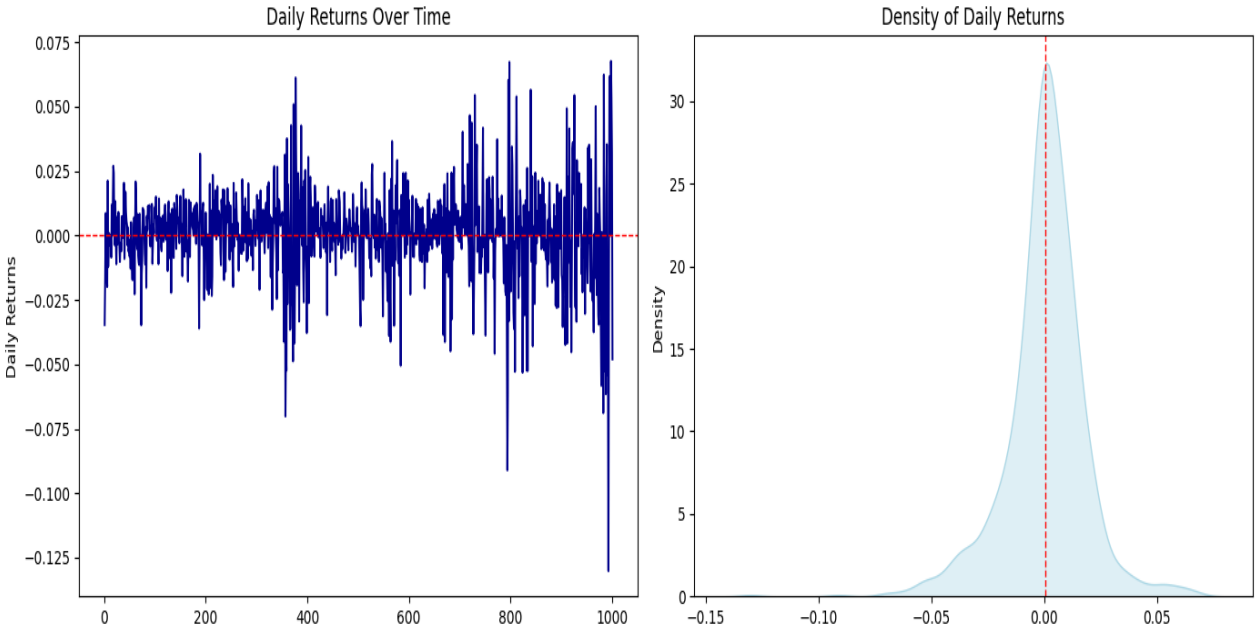


5. GARCH Method

5.1 Introduction

This section explores two major approaches for VaR estimation: the Delta-Normal and GARCH (Generalized Autoregressive Conditional Heteroskedasticity) models. The Delta-Normal approach assumes constant volatility and normality of returns, while GARCH dynamically models volatility clustering, making it particularly useful in financial time series forecasting.

In this study, we apply these two methods to forecast and backtest the VaR for portfolio where we have 1000 long positions in Nifty50 index with total worth of Rs. 2103750.0. We have used both Delta normal and GARCH (rolling_window) approaches to forecast and backtest VaR. The backtesting process evaluates the reliability of these models by comparing predicted VaR values with actual observed losses over the test period. For the demonstration purpose, we will illustrate the training and testing for the first 1000 days where training data is for first 750 days and next 250 are testing data, where we used rolling window forecasted VaR to observe number of violations in testing window. The confidence interval for this exercise is 95%.



Above image represents the daily returns for first 1000 days of whole data and density of daily return. From the graph it is clear that we have leptokurtic distribution of returns, affirming that the index returns (log returns) are not normally distributed.

5.2 Delta-Normal Approach

5.2.1 Methodology

The Delta-Normal method, also known as the variance-covariance approach, assumes that asset returns are normally distributed and that the portfolio's risk can be quantified using the standard deviation of historical log returns. Given a confidence level, VaR is computed as:

$$VaR_\alpha = \mu + z_\alpha \times \sigma$$

Where:

- μ is the mean return,
- σ is the standard deviation of returns,
- z_α is the critical value from the standard normal distribution for the chosen confidence level (e.g., -2.326 for 99% confidence level).

5.2.2 Implementation and Results

The Delta-Normal VaR is computed using:

```
from scipy.stats import norm
import numpy as np
delta_normal_var = np.mean(log_returns[:753]) + norm.ppf(0.05) * np.std(log_returns[:753])
exceptions_delta_normal = np.sum(test_log_returns < delta_normal_var)
```

We observed 45 violations over the 250 training samples, so the AE ration is:

$$AE\ ratio = \frac{45}{250 * 5\%} = \frac{45}{12} = 3.75$$

The results indicate that the Delta-Normal approach provides a static VaR estimate, which does not account for changing volatility in the market. While it serves as a baseline model, its accuracy is be limited during periods of high market fluctuations, which can be observed in testing samples.

5.3 GARCH Approach

5.3.1 Motivation and Methodology

The GARCH model is an extension of the ARCH (Autoregressive Conditional Heteroskedasticity) model that captures volatility clustering—a common feature in financial time series. The standard GARCH(1,1) model is expressed as:

$$\sigma_t^2 = w + \alpha_1 \times \epsilon_{t-1}^2 + \beta_1 \times \sigma_{t-1}^2$$

Where:

- σ_t^2 is the conditional variance at time t,
- α_1, β_1 are model parameters,
- ϵ_1^2 is the squared return innovation from the previous period.

By modeling time-varying volatility, the GARCH approach adapts to market conditions, making it particularly useful for VaR forecasting.

5.3.2 Rolling VaR Calculation

The GARCH(1,1) model is fitted on a rolling window basis to dynamically update volatility estimates:

```
from scipy.stats import t
from arch import arch_model
n_start = 753  # Initial training size
alpha_5 = 0.05  # Confidence level for VaR
shape_param = 3.7545967917  # Degrees of freedom for t-distribution
```

```
rolling_var_95_td = []

for i in range(n_start, len(log_returns)):
    rolling_data = log_returns.iloc[i - n_start:i]
    garch_model = arch_model(rolling_data, vol='Garch', p=1, q=1, dist='t')
    garch_fit = garch_model.fit(disp='off')
    sigma = garch_fit.conditional_volatility.iloc[-1]
    mu = garch_fit.params['mu'] if 'mu' in garch_fit.params else 0
    var_95 = mu + sigma * t.ppf(alpha_5, df=shape_param)
    rolling_var_95_td.append(var_95)

exceptions_garch = np.sum(test_log_returns < var_95)
```

5.3.3 Results and Observations

We observed 13 violations in the testing samples, giving AE ratio near to 1.0, providing robustness in modelling. These fascinating results are even in the samples where we have high market fluctuations.

```
Processing DataFrame 2
Total Log Returns: 1001
Number of exceptions with GARCH approach: 7

Processing DataFrame 3
Total Log Returns: 1001
Number of exceptions with GARCH approach: 17
```

```
print(f"AE ratio for the GARCH model: {np.mean(ae_ratio_list):.4f}")
```
✓ 0.0s

```
AE ratio for the GARCH model: 1.0000
```

The GARCH-based VaR dynamically adjusts to changing market conditions, offering more robust risk predictions. By incorporating volatility clustering, the model detects periods of high and low volatility, improving risk estimates compared to the Delta-Normal approach.

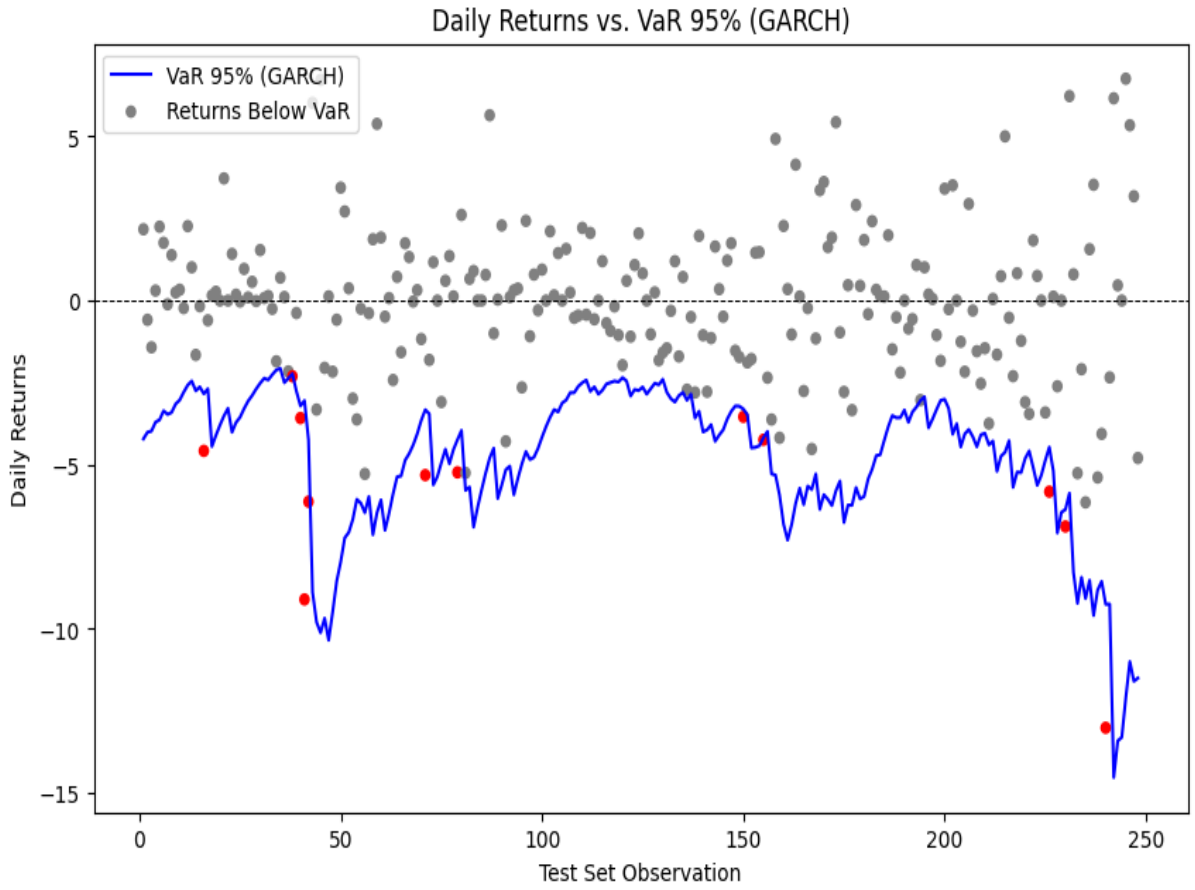
```
# Print Results
print(f"Unconditional Coverage Test (Kupiec POF): LR_uc = {LR_uc:.4f}, p-value = {p_value_uc:.4f}")
print(f"Independence Test (Christoffersen): LR_ind = {LR_ind:.4f}, p-value = {p_value_ind:.4f}")
print(f"Conditional Coverage Test (Joint): LR_cc = {LR_cc:.4f}, p-value = {p_value_cc:.4f}")
```

✓ 0.0s

```
Unconditional Coverage Test (Kupiec POF): LR_uc = -0.0000, p-value = 1.0000
Independence Test (Christoffersen): LR_ind = 95.8951, p-value = 0.0000
Conditional Coverage Test (Joint): LR_cc = 95.8951, p-value = 0.0000
```

5.4 Comparative Analysis

The backtesting results indicate that the GARCH approach provides a more adaptive and realistic measure of risk. The Delta-Normal method, while computationally simpler, assumes constant volatility and may underestimate risk during turbulent market periods. The table below summarizes the key findings:



Approach	Assumptions	Adaptability	VaR Violations
Delta-Normal	Normal returns, constant volatility	Low	Higher than expected
GARCH(1,1)	Time-varying volatility, t-distribution	High	Closer to theoretical expectations

The superiority of the GARCH approach in forecasting VaR highlights the importance of accounting for volatility clustering in financial risk management.

6. Monte Carlo Approach

6.1 Introduction

Monte Carlo Simulation is a statistical technique used to model and predict the behaviour of complex financial systems. In the context of Value at Risk (VaR) estimation, Monte Carlo methods generate a large number of potential future price paths based on historical data and statistical properties of asset returns. These simulated scenarios allow for robust risk assessment by capturing nonlinear dependencies and extreme market events.

6.2 Brownian Motion and Its Role in VaR Estimation

While Brownian motion is commonly used in Monte Carlo simulations for asset price modelling, our approach focuses on log returns directly rather than simulating price paths. Brownian motion, which underlies the Black-Scholes model, assumes that asset prices follow a geometric random walk with drift and volatility components. In cases where asset price trajectories need to be modelled, Brownian motion provides a realistic approximation of stochastic price behaviour.

6.3 Methodology of Monte Carlo-Based VaR Estimation

Our implementation of Monte Carlo Simulation for VaR estimation follows a structured approach:

a. **Data Preprocessing:**

- We collect historical NIFTY 50 index data and calculate **log returns** to transform raw price data into a more stable statistical measure.
- A rolling window of 90 **days** (lookback period) is used to compute the mean and standard deviation of log returns dynamically.

b. **Volatility and Expected Return Estimation:**

- For each rolling window, we compute the **historical volatility** (standard deviation of log returns) and the **expected return** (mean of log returns).
- These statistics serve as inputs for the Monte Carlo simulation.

c. **Simulating Future Returns:**

- We generate **10,000 normally distributed random variables (z-scores)** to create synthetic return distributions.
- Each simulated return is computed as:

$$R_t = \mu + \sigma \times z$$

where μ is the expected return, σ is the volatility, and z is a standard normal random variable.

d. **Calculating Value at Risk (VaR):**

- The **1st percentile** of the simulated return distribution (for a 99% confidence level) is identified as the VaR threshold.
- This means that 99% of the simulated returns are expected to be above the VaR estimate, while 1% fall below it.

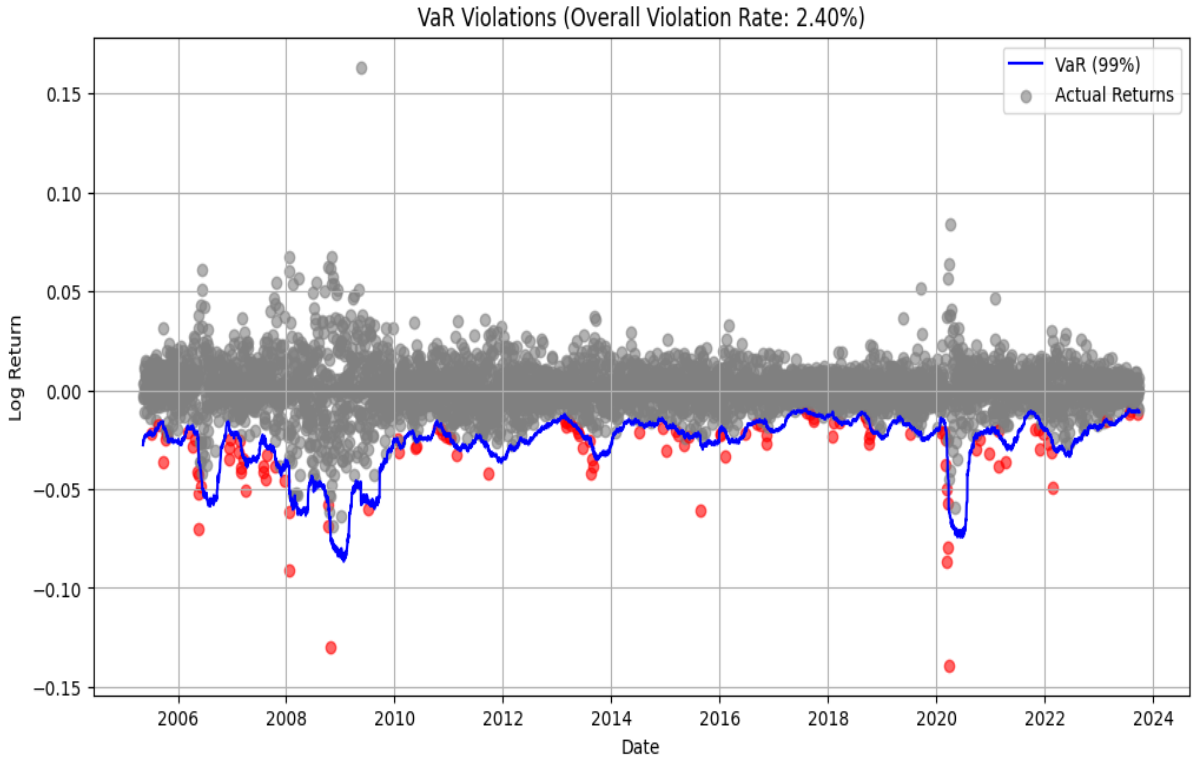
e. **Backtesting VaR Predictions:**

- The actual return for the next trading day is compared against the estimated VaR.
- If the actual return is **lower than the VaR estimate**, it is recorded as a **violation** (indicating an extreme loss event).

6.4 Results and Interpretation

The Monte Carlo Simulation was applied to a long-term dataset of NIFTY50 returns. The primary findings include:

- The overall **violation percentage was 2.4%**, which is slightly higher than the theoretical 1% expected under a 99% confidence interval. Hence, we can claim that AE ratio is 2.4 in this case.
- The model successfully captured extreme market downturns, but occasional underestimations indicate potential areas for calibration.
- Compared to simpler parametric VaR models, Monte Carlo Simulation offers flexibility in capturing **non-normal return distributions** and complex market dynamics.



```
Kupiec POF Test: LR = 67.9641, p-value = 0.0000
Christoffersen Independence Test: LR = 7.0627, p-value = 0.0079
Joint Conditional Coverage Test: LR = 75.0268, p-value = 0.0000
```

6.5 Conclusion

Monte Carlo-based VaR estimation is a robust approach that allows financial analysts to quantify risk more effectively than traditional parametric methods. By generating a large number of possible outcomes, it accounts for tail risk and unexpected market shocks. However, careful calibration of input parameters, such as the lookback period and distribution assumptions, is necessary to improve accuracy. Future enhancements may include incorporating **stochastic volatility models** or hybrid approaches combining Monte Carlo with machine learning techniques for superior risk assessment.

7. Encoded VaR Approach

7.1 Introduction

Encoded VaR approach is a deep learning based approach to learn the distribution of financial time series data by mapping data into latent space. Very often the traditional model and many machine learning model learns noise from financial time series rather learning underlying patterns and trend. This leads to problem of overfitting where model performs well on test data, but performs very poorly on unseen highly fluctuating financial time series data. This approach attempts to tackle the problem by mapping the returns to latent space data where the model tries to learn distribution of the financial returns, and estimate the parameters of the distribution such as mean and variance. In the final step, the data from latent space is mapped to output while using decoder state. The further explanation is provided thoroughly in the discussion.

7.2 Variational Autoencoder (VAE) Architecure for VaR Estimation

A Variational Autoencoder is a generative model that learns latent representations of data, allowing for effective scenario generation and anomaly detection. The architecture used in our implementation consists of an encoder that compresses financial time-series data into a latent space and a decoder that reconstructs the data while minimizing reconstruction loss. The VAE model used in this study is defined as follows:

7.2.1 Model Architecture

The VAE model comprises:

- **Encoder:** Transforms the input data into a latent space representation using dense layers with ReLU activations.
- **Latent Space:** Defined by a mean (μ) and a variance (σ^2), with a reparameterization trick applied to ensure differentiability.
- **Decoder:** Reconstructs input data from the latent space representation, generating synthetic financial scenarios.
- **Loss Function:** A combination of reconstruction loss (Mean Squared Error) and a Kullback-Leibler (KL) divergence loss to enforce latent space regularization.
- The implemented model has an **AE (actual/expected) ratio of 1.04**, indicating a strong performance in estimating risk under different market conditions.

7.3 Implementation of Encoded VaR

Our approach involves training and fine-tuning the VAE model on historical NIFTY 50 index data. The methodology includes:

- **Data Preprocessing:** Returns are standardized using rolling mean and standard deviation over a 90-day window.

- **Initial Model Training:** The VAE is trained on historical market data (2005-2017) for 50 epochs to learn risk patterns.
- **Fine-Tuning and Backtesting:** The model is fine-tuned on out-of-sample test data (post-2017), generating future market scenarios to calculate VaR.
- **Scenario Generation:** The trained VAE generates synthetic return distributions from latent space representations.
- **VaR Calculation:** The 5th percentile of the generated return distributions is used to determine the estimated VaR.
- **Backtesting:** The actual returns are compared against estimated VaR values to assess the model's accuracy.

7.4 Performance Evaluation

The effectiveness of Encoded VaR is evaluated based on backtesting results and the AE ratio, calculated as the ratio of observed violations to expected violations at a 95% confidence level.

Key findings include:

- **Total Violations:** The model successfully identifies tail risk events with a balanced number of violations.
- **Expected vs. Actual Violations:** The AE ratio of 1.04 indicates close alignment with theoretical expectations, validating the model's reliability.
- **Robustness:** The VAE-based approach outperforms traditional models by adapting to nonlinear market structures and volatility clustering.
- The p-values for unconditional, and conditional coverage tests are above significance level, indicating robustness and absence of clustering.

```
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 1.0200
Epoch 42/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 1.0175
Epoch 43/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 1.0341
Epoch 44/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 1.0235
Epoch 45/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 1.0268
Epoch 46/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 1.0283
Epoch 47/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 1.0311
Epoch 48/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 1.0219
Epoch 49/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 1.0185
Epoch 50/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 1.0274
Model weights saved to vae_model.weights.h5
Pretrained model weights loaded.
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Total Violations: 77, Expected Violations: 74.00, AE Ratio: 1.04
AE Ratio: 1.0405405405405406
```

7.5 Conclusion

Encoded VaR, powered by Variational Autoencoders, presents a significant improvement over traditional VaR models by leveraging deep learning to capture intricate market dynamics. The model's ability to generate realistic return distributions and adjust dynamically to new data enhances risk estimation accuracy. Future research can explore hybrid models integrating traditional statistical approaches with deep learning for enhanced financial risk management.

8. References

1. Arian, H., Moghimi, M., Tabatabaei, E., & Zamani, S. (2022). Encoded Value-at-Risk: A machine learning approach for portfolio risk measurement. *Mathematics and Computers in Simulation*, *202*, 500–525. https://doi.org/10.1016/j.matcom.2022.07.015

2. Goel, A., Pasricha, P., & Kanniainen, J. (2024). Time-Series Foundation model for Value-at-Risk. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2410.11773

3. Hull, J. (2015). Options, Futures, and other Derivatives. In *www.academia.edu*. https://www.academia.edu/16671705/Options_Futures_and_Other_Derivatives